

UNIVERSIDADE FEDERAL DE ITAJUBÁ

Engenharia de Dados na Prática

Ambiente de trabalho e arquitetura em camadas

Profa. Bárbara Pimenta Caetano

O que veremos hoje

- 1 Por que versionar o seu código
- 2 Git e GitHub na prática
- 3 O exemplo que vamos usar no semestre
- 4 A arquitetura em três camadas
- 5 Passo a passo: montar o projeto juntos

Você já viu esta pasta

```
analise.py  
analise_v2.py  
analise_final.py  
analise_final_CORRIGIDO.py  
analise_final_CORRIGIDO_ok.py
```

Qual delas roda? Qual gerou o gráfico que você mandou por e-mail?

Controle de versão existe para que essa pasta nunca precise existir.

Git e GitHub não são a mesma coisa

Git

O programa que roda na sua máquina e guarda o histórico do projeto.

Funciona sem internet

GitHub

O serviço na nuvem onde você publica esse histórico, com cópia de segurança.

Funciona como vitrine

Dá para usar Git sem GitHub. Não dá para usar GitHub sem Git.

O fluxo que você vai repetir toda aula

`git add`

Seleciona o que entra
na próxima versão



`git commit`

Registra a versão com
uma mensagem



`git push`

Envia a versão para o
GitHub

A mensagem do commit é para o você de daqui a três meses. Escreva o que mudou.

O exemplo que vamos usar

Artistas mais ouvidos no Spotify

500 artistas, 14 colunas. Nome, país de origem, idioma, gênero, ano de estreia e número de streams em milhões, separado entre streams como artista principal e como participação.

Esse é o exemplo das aulas. Seu projeto usa a sua própria fonte de dados.

Kaggle. Most Streamed Artists on Spotify: <https://www.kaggle.com/datasets/rishavsvault/most-streamed-artists-on-spotify>

A cara do arquivo que vamos receber

```
Artist Name, Sex, Country of Origin, ...  
Drake, Male, Canada, English, Hip-Hop, ...  
Taylor Swift, Female, United States, ...
```

- Nomes de coluna com espaços, parênteses e sinal de porcentagem
- A coluna Artist Type vem com um espaço sobrando no início do nome
- Percentuais com oito casas decimais

Nada disso se conserta no arquivo. Conserta-se na camada seguinte.

A estrutura do projeto na sua máquina

```
meu-projeto/  
  dados/  
    bronze/  
    prata/  
    ouro/  
  src/  
  README.md  
  .gitignore
```

As três camadas são pastas.
Nada de mistério.

O arquivo .gitignore diz ao Git
o que ele deve deixar de fora.

O que vai para o GitHub

O critério não é dado ou código. É o que se consegue reconstruir.

Vai

O código do projeto

O README e o .gitignore

O dado bruto, se for pequeno

Não vai

O que o seu código escreve

O ambiente virtual

Senhas e chaves de acesso

**A bronze é a única coisa que o código não regenera se a fonte sair do ar.
Por isso ela sobe, desde que caiba.**

Por que não fazer tudo de uma vez

- Se o tratamento estiver errado, você refaz sem baixar o dado de novo
- Cada etapa pode ser conferida separadamente
- Você consegue mostrar de onde veio cada número

Esse padrão se chama arquitetura medallion e é o mesmo usado no mercado.

Databricks. What is a medallion architecture? Adotado também pelo Microsoft Fabric e pelo Azure.

Camada bronze

- O dado exatamente como veio da fonte

```
dados/bronze/artists.csv
```

O arquivo do Kaggle, com os nomes de coluna originais e os percentuais com oito casas.

Regra: a bronze nunca é editada.

Camada prata

- O dado limpo, padronizado e integrado

```
dados/prata/artistas.csv
```

Colunas renomeadas para artist_name e total_streams_mi, tipos corrigidos e percentuais arredondados.

Regra: toda transformação vira código.

Camada ouro

- O dado modelado para responder perguntas

```
dados/ouro/streams_por_pais.csv
```

Total de streams por país e por gênero, pronto para alimentar o painel final.

Regra: a ouro existe para um uso definido.

As três camadas do seu projeto



Bronze

`artists.csv`



Prata

`artistas.csv`



Ouro

`streams_por_pais.
csv`

O dado nunca volta. Ele só avança de uma camada para a seguinte.



Agora vamos fazer juntos

Doze passos, do zero até o primeiro commit publicado.

PASSO 1

Conferir o que está instalado

Abra o VS Code e o terminal integrado com Ctrl e crase. Rode:

```
python --version  
git --version  
code --version
```

Se algum comando não for reconhecido, chame a professora antes de seguir.

PASSO 2

Criar a pasta do projeto

```
mkdir meu-projeto  
cd meu-projeto  
mkdir dados dados/bronze dados/prata dados/ouro src  
git init
```

O comando `git init` transforma a pasta em um repositório.

Por que ambiente virtual

Sem ambiente virtual, todo projeto da sua máquina divide as mesmas bibliotecas.

Um projeto pede pandas 1.5, outro pede 2.2

Você instala vinte bibliotecas ao longo do ano

O colega clona o seu repositório

- › Você não consegue atender aos dois
- › Não sabe mais quais o projeto realmente usa
- › Não faz ideia do que precisa instalar

O ambiente virtual dá a cada projeto a sua própria caixa de bibliotecas.

PASSO 3

Criar e ativar o ambiente virtual

Criar, uma única vez:

```
python -m venv .venv
```

Ativar, toda vez que abrir o projeto. Windows:

```
.venv\Scripts\Activate.ps1
```

Linux ou macOS:

```
source .venv/bin/activate
```

PASSO 4

Declarar e instalar as dependências

Crie o arquivo requirements.txt na raiz, com uma biblioteca por linha:

```
pandas
```

Instale a partir dele:

```
pip install -r requirements.txt
```

Esse arquivo é o que permite outra pessoa reproduzir o seu ambiente com um único comando.

PASSO 5

Criar o repositório no GitHub

- Entre em github.com
- Clique em New repository
- Mesmo nome da pasta
- Deixe como Public

Não marque a opção de criar README. Ele já existe na sua pasta.

Create a new repository

esquema da tela

Repository name

Description (optional)

☒ Public ☐ Private

☐ Add a README file **◀ deixe desmarcado**

Create repository

PASSO 6

Ligar a sua pasta ao GitHub

Copie o endereço que o GitHub mostra e rode os comandos, trocando pelo seu:

Quick setup	<i>esquema da tela</i>
HTTPS	github.com/voce/meu-projeto.git

```
git remote add origin URL_COPIADA  
git branch -M main
```

Esse endereço aparece assim que o repositório é criado, na tela seguinte.

PASSO 7

Criar o arquivo .gitignore

Crie um arquivo chamado .gitignore na raiz do projeto:

```
dados/prata/*  
dados/ouro/*  
!dados/**/.gitkeep  
.venv/  
__pycache__/  
kaggle.json
```

A pasta bronze não aparece aqui. Ela sobe junto com o código.

O ambiente virtual fica de fora porque o requirements o reconstrói.

PASSO 8

Criar os arquivos .gitkeep

O Git não versiona pasta vazia. Sem esse arquivo, as camadas somem do GitHub.

Windows, no PowerShell:

```
ni dados/bronze/.gitkeep, dados/prata/.gitkeep,  
dados/ouro/.gitkeep
```

Linux ou macOS:

```
touch dados/bronze/.gitkeep dados/prata/.gitkeep \  
dados/ouro/.gitkeep
```

São arquivos vazios. Existem apenas para a pasta poder ser versionada.

PASSO 9

Abrir o arquivo e medir

Coloque o CSV em dados/bronze e crie src/ler_dados.py:

```
import pandas as pd

CAMINHO = "dados/bronze/artists.csv"

df = pd.read_csv(CAMINHO)

print(df.shape)
```

Primeira pergunta: quantas linhas e quantas colunas o arquivo tem?

PASSO 10

Olhar os nomes das colunas

```
import pandas as pd
CAMINHO = "dados/bronze/artists.csv"
df = pd.read_csv(CAMINHO)
print(df.shape)

for coluna in df.columns:
    print(f'"{coluna}"')
```

As aspas são de propósito. Elas revelam espaços que você não veria de outro jeito.

PASSO 11

Conferir se as colunas esperadas existem

...

```
ESPERADAS = ["Artist Name", "Artist Type",  
             "Debut Year"]
```

```
faltando = [c for c in ESPERADAS  
            if c not in df.columns]
```

```
print("Nao encontradas:", faltando)
```

A saída acusa uma coluna que existe no arquivo. Por que ela não foi encontrada?

PASSO 12

Organizar em funções

```
def carregar() :  
    return pd.read_csv(CAMINHO)  
  
def conferir_estrutura(df) :  
    print(df.shape)  
    print(df.dtypes)  
  
if __name__ == "__main__":  
    conferir_estrutura(carregar())
```

Um script solto vira um módulo reaproveitável. É assim que o pipeline começa.

O que você acabou de fazer



Hoje: estrutura

O arquivo tem o formato previsto?

As colunas têm os nomes esperados?

Os tipos vieram certos?



Nas próximas aulas: conteúdo

Quantos valores estão faltando?

Como os valores se distribuem?

Existem outliers e duplicatas?

São perguntas diferentes. A de hoje você respondeu escrevendo código. A outra, mais adiante, uma ferramenta responde por você.

ÚLTIMO PASSO

Primeiro commit e publicação

```
git add .  
git commit -m "Estrutura inicial"  
git push -u origin main
```

Confira que a pasta .venv não subiu. Se ela aparecer, o .gitignore está errado.

voce / meu-projeto		esquema da tela
dados	Estrutura inicial	
src	Estrutura inicial	
.gitignore	Estrutura inicial	
README.md	Estrutura inicial	

Na próxima aula isso vira código

Hoje você colocou o arquivo na pasta com a mão. Na próxima aula o próprio script busca o dado:

```
import kagglehub

caminho = kagglehub.dataset_download(
    "rishavsvault/most-streamed-artists-on-spotify"
)

print(caminho)
```

É assim que a camada bronze passa a ser reconstruída por qualquer pessoa que clone o repositório.

Entrega de hoje

O link do seu repositório, com a estrutura de pastas, o .gitignore, o README e o primeiro commit publicado.

Na próxima aula: trazer o seu dado para dentro do projeto e construir a sua camada bronze.